

Universidad Diego Portales

Facultad de Ingeniería y Ciencias

Curso Arquitectura de Software

Sistema de Gestión de Inventario Omnicanal (Sync-Stock)

Informe Final - 3ra Entrega

Organización: Tienda de Pesca "PescaTotal"

Área: Departamento de Ventas e Inventario

Responsables: Fernando Llancao, Carlos Jara, Bastian Peña, Nicolas Canelo

Fecha: 7 de Junio de 2026

Ubicación: Región Metropolitana, Chile

Semestre 1/2026

Índice

1. Equipo de Trabajo	3
2. Descripción del Sistema y Contexto Organizacional	3
2.1. La Organización: PescaTotal	3
2.2. Área de Trabajo: Departamento de Ventas e Inventario	3
2.3. Problemática Actual	3
2.4. Sistema Propuesto: Sync-Stock	4
3. Objetivos y Perfiles de Usuario	4
3.1. Objetivos del Sistema	4
3.1.1. Objetivo General	4
3.1.2. Objetivos Específicos	4
3.2. Perfiles de Usuario	4
4. Requerimientos Funcionales	5
5. Modelo de Datos y Mecanismo de Persistencia	7
5.1. Mecanismo de Persistencia y Decisiones de Diseño	7
5.2. Diccionario de Datos (Entidades del Sistema Omnicanal)	7
6. Arquitectura SOA y Mediación a través de un Bus de Servicios (ESB)	8
6.1. El Rol del Enterprise Service Bus (ESB)	8
6.2. Mecanismos Clásicos de Mediación SOA Implementados	10
6.3. Componentes de la Arquitectura	10
6.3.1. Componentes Cliente (Consumidores de Servicios)	10
6.3.2. Componentes de Servicio (Proveedores del Bus)	10
7. Interfaces de los Componentes y Contratos Técnicos	11
7.1. Detalle de Contratos Técnicos de la API	11
7.1.1. 1. Autenticación de Usuarios (Auth Service)	11
7.1.2. 2. Reserva Temporal de Stock (Inventory Service)	13
7.1.3. 3. Confirmación de Reserva (Inventory Service)	13
7.1.4. 4. Consulta de Stock en Tiempo Real (Inventory Service)	14
8. Documentación de Servicios y Procesos Cliente	15
8.1. Registro de Nuevos Productos	15
8.2. Modificación de Producto	16
8.3. Despacho de Órdenes Consolidadas	16
8.4. Redirección General a través del Bus de Servicios (GET)	17
9. Documentación del Sistema y Evidencias de Funcionamiento	18
9.1. Explicación Técnica del Código e Implementación	18
9.2. Evidencias de Funcionamiento y Logs	18

10. Análisis Crítico	20
10.1. Latencia y Concurrencia de los Bloqueos Pesimistas	20
10.2. Desafíos del Desarrollo de un ESB Personalizado TCP	20
10.3. Seguridad y RBAC en Entornos Distribuidos	20
10.4. Control de Inactividad y Seguridad Física en Tienda	20
11. Conclusiones	21

1. Equipo de Trabajo

Para el desarrollo del sistema Sync-Stock, se ha conformado un equipo de trabajo liderado por el responsable del área de desarrollo:

Integrante	Cargo	Responsabilidades
Fernando Llanca	CEO / Líder de Proyecto	■ Liderazgo estratégico y toma de decisiones ejecutivas. ■ Coordinación directa con los dueños de PescaTotal. ■ Supervisión general del análisis y diseño del sistema.
Nicolas Canelo	Arquitecto de Software	■ Diseño de la estructura técnica y componentes. ■ Selección del stack tecnológico adecuado para tiempo real.
Carlos Jara	Analista de Sistemas	■ Levantamiento y especificación de requerimientos. ■ Definición del modelo de datos y persistencia.
Bastian Peña	Desarrollador Full-stack	■ Implementación de la lógica de negocio y API. ■ Desarrollo de la interfaz de usuario amigable.

2. Descripción del Sistema y Contexto Organizacional

2.1. La Organización: PescaTotal

PescaTotal es una tienda retail especializada en artículos de pesca recreativa y equipamiento outdoor. A pesar de ser una organización de escala pequeña, posee un flujo de ventas constante y una base de clientes fidelizada en la Región Metropolitana.

2.2. Área de Trabajo: Departamento de Ventas e Inventario

Esta unidad operativa gestiona el stock físico de la tienda y, recientemente, la administración del nuevo canal e-commerce. Sus funciones incluyen la recepción de suministros, atención al público y preparación de pedidos online.

2.3. Problemática Actual

La tienda enfrenta dificultades operacionales críticas derivadas de la falta de integración digital:

- **Incumplimiento de pedidos y fricción con clientes por falta de gestión de reservas:** Al carecer de un mecanismo que bloquee o separe físicamente los productos apenas se venden por el canal e-commerce, los artículos quedan expuestos en las vitrinas y son comprados por clientes presenciales. Esto genera un problema grave para el negocio: la necesidad de cancelar compras online ya pagadas, procesar devoluciones de dinero, sobrecargar al personal de atención al cliente y generar un impacto negativo en la reputación y fidelidad de los compradores de PescaTotal.

2.4. Sistema Propuesto: Sync-Stock

Sync-Stock es una plataforma segura de gestión de inventario omnicanal. Su objetivo principal es actuar como un núcleo de datos centralizado que sincronice el stock en tiempo real entre la tienda física y el e-commerce.

Para garantizar la integridad de las operaciones, el sistema contará con acceso restringido mediante roles, entregando a cada empleado (Vendedor, Encargado E-commerce o Administrador) solo las herramientas que necesita. Al registrar una venta en el mesón o recibir un pedido web, el sistema no solo actualiza la disponibilidad global inmediatamente, sino que genera un estado de Reserva inmutable para las compras web, bloqueando la posibilidad de sobreventas en la tienda física. Esto asegurará la coherencia total de los datos y protegerá la experiencia de compra de los clientes.

3. Objetivos y Perfiles de Usuario

3.1. Objetivos del Sistema

3.1.1. Objetivo General

Implementar un sistema integral que garantice la coherencia total del inventario entre la tienda física y el e-commerce de PescaTotal, optimizando la eficiencia operativa y eliminando errores de sobreventa.

3.1.2. Objetivos Específicos

1. Sincronizar de forma automática y bidireccional el stock tras cada transacción.
2. Implementar un módulo de reserva inmediata para asegurar productos vendidos vía web.
3. Proporcionar visibilidad de stock real al personal de ventas mediante una interfaz simple.
4. Reducir en un 85 % las horas-hombre semanales dedicadas a la conciliación y revisión manual de inventarios, alcanzando esta métrica al finalizar el segundo mes de operación en producción del sistema Sync-Stock.”

3.2. Perfiles de Usuario

- **Usuario Primario (Vendedor):** Utiliza el sistema en el mesón de la tienda para consultar disponibilidad y cerrar ventas presenciales.

- Frecuencia de uso: Permanente durante la jornada.
- Nivel tecnico: Intermedio.
- **Usuario Secundario (Encargado E-commerce):** Gestiona los pedidos web, visualiza las reservas y confirma la salida física del producto.
 - Frecuencia de uso: Permanente durante la jornada.
 - Nivel tecnico: Intermedio.
- **Usuario de Apoyo (Administrador):** Gestiona el catálogo, permisos de usuarios y configuraciones de alertas de stock.
 - Frecuencia de uso: Variable, segun necesidades de la tienda.
 - Nivel tecnico: Avanzado.

4. Requerimientos Funcionales

Código	Nombre	Descripción
RF-001	Sincronización Real-Time	El sistema debe descontar el stock del catálogo online inmediatamente tras una venta física confirmada.
RF-002	Reserva de Stock Online	Al detectarse una orden de compra web, el sistema debe mover el producto al estado Reservado”, restándolo del stock físico vendible.
RF-003	Gestión de Productos	Permitir el registro, edición y categorización de artículos con códigos de barra y ubicación física.
RF-004	Control de Despachos	Funcionalidad para confirmar la salida de productos reservados, actualizando el estado a ”Vendido” definitivamente.
RF-005	Alertas de Quiebre	Notificaciones automáticas cuando el stock disponible llegue a un umbral mínimo configurado.
RF-006	Log de Auditoría	Registro detallado de quién, cuándo y por qué canal se realizó cada movimiento de inventario.
RF-007	Autenticación de Usuarios (Login)	El sistema debe requerir que todo usuario inicie sesión mediante credenciales (correo/usuario y contraseña) antes de acceder a cualquier funcionalidad.
RF-008	Control de Acceso Basado en Roles (RBAC)	El sistema debe restringir las funcionalidades e interfaces de acuerdo con el rol del usuario (Administrador, Vendedor, Encargado E-commerce). Por ejemplo, solo el Administrador podrá gestionar permisos o el catálogo.
RF-009	Cierre de Sesión y Tiempos de Inactividad	El sistema debe permitir el cierre de sesión manual y forzar un cierre automático tras 30 minutos de inactividad por motivos de seguridad en los equipos compartidos de la tienda.

5. Modelo de Datos y Mecanismo de Persistencia

Para dar soporte a las funcionalidades del sistema Sync-Stock en un entorno retail omnicanal real, se ha diseñado una estrategia de persistencia transaccional de alta consistencia y un modelo de datos relacional detallado.

5.1. Mecanismo de Persistencia y Decisiones de Diseño

Se ha optado por implementar una **Base de Datos Relacional (RDBMS)** utilizando **PostgreSQL**. Esta decisión responde a la necesidad de resolver problemas complejos de concurrencia y consistencia que un sistema omnicanal enfrenta en tiempo real:

1. **Consistencia Transaccional Estricta (ACID):** Para evitar la sobreventa, los cambios en el inventario deben procesarse con aislamiento transaccional estricto. PostgreSQL permite aislar operaciones mediante el nivel de aislamiento de transacciones *Read Committed* combinado con bloqueos pesimistas explícitos.
2. **Estrategia de Concurrencia y Bloqueos Pesimistas (Pessimistic Locking):** Al procesar una compra en el Punto de Venta (POS) o iniciar un proceso de reserva web, el sistema ejecuta una transacción que bloquea las filas correspondientes de la tabla de inventario mediante la sentencia `SELECT ... FOR UPDATE`.
 - Esto garantiza que ningún otro proceso pueda modificar la cantidad del producto hasta que la transacción actual finalice (haga *COMMIT* o *ROLLBACK*).
 - **Prevención de Deadlocks:** Para evitar bloqueos mutuos (*deadlocks*) bajo cargas de alta concurrencia, el sistema ordena siempre los ítems de la consulta de forma determinista (ordenados por el identificador del producto) antes de solicitar el bloqueo.
3. **Política de Expiración de Reservas y Liberación de Stock (Time-To-Live):** El canal e-commerce genera reservas temporales para proteger el stock de un cliente mientras completa el pago. Sin embargo, para evitar que el stock quede bloqueado indefinidamente si el cliente abandona el carrito, se implementa una regla de expiración de 15 minutos.
 - Cada reserva se registra en la tabla `Reserva` con un campo `FechaExpiracion`.
 - Un proceso daemon (o *cron job*) en el bus de servicios ejecuta periódicamente (cada 1 minuto) una transacción que selecciona las reservas expiradas en estado `Pendiente`, revierte el stock reservado al stock disponible (`StockDisponible = StockDisponible + Cantidad`) y marca la reserva como `Expirada`.
4. **Pooling de Conexiones y Alta Disponibilidad:** Para asegurar tiempos de respuesta menores a 200 ms y soportar picos de carga concurrentes entre la tienda y el sitio web, se utiliza un pooler de conexiones (**PgBouncer**) configurado en modo transacción. Esto permite optimizar el uso de recursos y mantener la latencia en rangos óptimos.

5.2. Diccionario de Datos (Entidades del Sistema Omnicanal)

A continuación se detalla el esquema de base de datos extendido que soporta la lógica del inventario multicanal, reservas y auditoría de transacciones:

Entidad	Atributo	Tipo de Dato	Descripción
Producto	ID_Producto (PK)	UUID	Identificador universal único del artículo.
	SKU	VARCHAR(50)	Código SKU único (Stock Keeping Unit).
	Nombre	VARCHAR(150)	Nombre o descripción del artículo.
	Precio	DECIMAL(12,2)	Precio de venta unitario.
Ubicacion	ID_Ubicacion (PK)	UUID	Identificador único de la ubicación de stock.
	Nombre	VARCHAR(100)	Nombre (ej. 'Bodega Central', 'POS Tienda').
	Tipo	ENUM	Tipo de instalación: 'Tienda' o 'Bodega'.
Inventario	ID_Inventario (PK)	UUID	Identificador del registro de inventario.
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Ubicacion_ID (FK)	UUID	Referencia a la entidad Ubicacion .
	StockDisponible	INT	Unidades físicas disponibles para venta inmediata.
	StockReservado	INT	Unidades retenidas temporalmente por e-commerce.
	StockTransito	INT	Unidades en proceso de traslado entre bodegas.

Cuadro 1: Diccionario de datos: Catálogo, Ubicaciones e Inventario.

6. Arquitectura SOA y Mediación a través de un Bus de Servicios (ESB)

Para resolver la integración y sincronización en tiempo real de múltiples canales, el sistema se ha estructurado bajo el patrón **SOA (Service-Oriented Architecture)** incorporando un **Enterprise Service Bus (ESB)** central como columna vertebral de comunicación. El ESB no solo actúa como canal de mensajería (utilizando un broker de mensajería como RabbitMQ), sino que implementa componentes de mediación clásicos que desacoplan físicamente y lógicamente a los clientes y servicios.

6.1. El Rol del Enterprise Service Bus (ESB)

En lugar de depender de llamadas HTTP directas y acopladas punto a punto entre la aplicación web e-commerce, el POS físico y los servicios de backend, el ESB centraliza las interacciones y proporciona las siguientes capacidades de infraestructura:

- **Garantía de Entrega y Asincronía:** Permite que las órdenes del canal digital ingresen a colas de mensajería seguras. Si el servicio de inventario o el sistema externo del courier de despachos sufren una caída temporal, las solicitudes permanecen encoladas garantizando que ningún pedido se pierda.
- **Desacoplamiento de Protocolos:** El bus permite conectar componentes heterogéneos, permitiendo que clientes web interactúen de manera sincrónica (REST/JSON)

Entidad	Atributo	Tipo de Dato	Descripción
Reserva	ID_Reserva (PK)	UUID	Identificador único de la reserva temporal.
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Ubicacion_ID (FK)	UUID	Referencia a la entidad Ubicacion .
	Cantidad	INT	Cantidad de unidades bloqueadas.
	Estado	ENUM	Estado de la reserva: 'Pendiente', 'Confirmada', 'Expirada'.
	FechaCreacion	TIMESTAMP	Marca de tiempo de la creación de la reserva.
	FechaExpiracion	TIMESTAMP	Límite temporal para consolidar la transacción.
Orden	ID_Orden (PK)	UUID	Identificador único de la orden de venta.
	Canal	ENUM	Origen del pedido: 'POS_Fisico' o 'E-Commerce'.
	Estado	ENUM	Estado del pedido: 'Creada', 'Pagada', 'Despachada', 'Cancelada'.
	FechaCreacion	TIMESTAMP	Marca de tiempo de registro de la orden.
	Total	DECIMAL(12,2)	Monto total de la transacción comercial.
LineaOrden	ID_Linea (PK)	UUID	Identificador de la línea de detalle de la orden.
	Orden_ID (FK)	UUID	Referencia a la entidad Orden .
	Producto_ID (FK)	UUID	Referencia a la entidad Producto .
	Cantidad	INT	Cantidad de unidades compradas.
	PrecioUnitario	DECIMAL(12,2)	Precio facturado para este producto.

Cuadro 2: Diccionario de datos: Reservas temporales y Órdenes de venta.

Entidad	Atributo	Tipo de Dato	Descripción
HistorialInv	ID_Historial (PK)	UUID	Identificador único para auditoría de movimientos.
	Producto_ID (FK)	UUID	Referencia al producto afectado.
	Ubicacion_ID (FK)	UUID	Ubicación física del movimiento.
	Cantidad	INT	Unidades modificadas (valor positivo o negativo).
	TipoMovimiento	ENUM	Tipo de evento: 'Venta', 'Reserva_Creada', 'Reserva_Expirada', 'Recepcion', 'Ajuste'.
	FechaHora	TIMESTAMP	Fecha y hora exacta de la transacción.
	Usuario_ID (FK)	UUID	Identificador del operario responsable.

Cuadro 3: Diccionario de datos: Historial de inventario para log de auditoría.

mientras que la comunicación interna y con sistemas legados se realice asíncronamente (AMQP/gRPC).

6.2. Mecanismos Clásicos de Mediación SOA Implementados

El ESB actúa como una capa de mediación activa, aplicando patrones de integración empresarial (EIP) para procesar los mensajes en tránsito:

1. **Enrutamiento Basado en Contenido (Content-Based Router):** Cuando el ESB recibe un mensaje de orden, inspecciona el campo `canal` del payload.
 - Si proviene de `E.Commerce`, enruta el mensaje hacia la cola de validación de reservas y pasarela de pago.
 - Si proviene de `POS_Fisico`, el mensaje es enrutado directamente al canal de facturación local e impresión de boleta física.
2. **Transformación y Traducción de Datos (Data Transformer):** Los canales de venta externos (como Shopify o un POS heredado) envían estructuras de datos propietarias. La capa de mediación del ESB traduce estos formatos a un esquema de datos estándar del sistema (denominado *Modelo Canónico*) antes de que ingresen a los servicios core de inventario.
3. **Enriquecimiento de Mensajes (Message Enricher):** Cuando una orden es capturada por el bus con información básica (ej. SKU y cantidad), el mediador consulta dinámicamente al servicio de catálogo para inyectar datos adicionales (descripción, peso para el courier, precio unitario vigente) antes de despachar el payload completo al servicio de persistencia y despacho.
4. **Puente de Protocolos (Protocol Bridge):** El ESB traduce las peticiones HTTP REST síncronas provenientes del frontend POS en mensajes AMQP dirigidos al backend, y viceversa, permitiendo que el sistema se comunique eficientemente sin acoplar tecnológicamente sus extremos.

6.3. Componentes de la Arquitectura

Bajo este esquema de mediación, los componentes del ecosistema SOA se dividen en:

6.3.1. Componentes Cliente (Consumidores de Servicios)

- **Cliente Web POS (Punto de Venta):** Interfaz local en la tienda física para vendedores. Consume servicios a través del ESB de forma síncrona para consultas de stock rápidas y de forma asíncrona para el cierre de caja.
- **Panel de Administración (Dashboard):** Aplicación web para el encargado de e-commerce y administradores. Permite monitorear el flujo de colas en el ESB, verificar alertas de stock y gestionar reservas en tiempo real.

6.3.2. Componentes de Servicio (Proveedores del Bus)

- **Auth Service:** Servicio transversal expuesto en el bus para verificar credenciales y gestionar roles de acceso (RBAC).
- **Inventory Service:** Servicio core que gestiona el inventario omnicanal físico y web, respondiendo a los comandos de reserva y confirmación enviados por el ESB.

- **Sales & Order Service:** Orquesta el ciclo de venta, consumiendo el Inventory Service mediante el ESB. Si una venta web falla en la pasarela de pago, este servicio coordina la compensación (cancelación de reserva).

7. Interfaces de los Componentes y Contratos Técnicos

Para garantizar la interoperabilidad del sistema Sync-Stock en un entorno omnicanal crítico en tiempo real, se definen contratos técnicos rigurosos. A continuación, se resume la relación entre interfaces, Requerimientos Funcionales (RF) y Requerimientos No Funcionales (RNF):

Servicio	Endpoint / Operación	RF Satisfecho	RNF Mapeado
Auth Service	POST /api/v1/auth/login	RF-007, RF-008	Seguridad, Expiración (30 min)
Inventory Service	POST /api/v1/inventory/reserve	RF-002	Consistencia Estricta, Alta Concurrencia
Inventory Service	POST /api/v1/inventory/commit	RF-004, RF-006	Integridad Transaccional, Auditoría
Inventory Service	GET /api/v1/inventory/stock/{sku}	RF-001, RF-003	Baja Latencia (< 200 ms)

Cuadro 4: Resumen de endpoints de servicios, RFs y RNFs.

7.1. Detalle de Contratos Técnicos de la API

A continuación se especifican las estructuras de las peticiones, las respuestas de éxito y el control de errores detallado (en formato JSON) para cada interfaz.

7.1.1. 1. Autenticación de Usuarios (Auth Service)

Endpoint: POST /api/v1/auth/login

Descripción: Valida las credenciales del operario y devuelve un token firmado (JWT) con su rol asociado (RBAC).

Estructura de la Petición (Request):

```
{
  "username": "fllancao",
  "password": "mi_clave_segura_123"
}
```

Respuesta de Éxito (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJhMWIy...[truncado]",
}
```

```
"usuario": {  
  "id": "e4b2d288-43ea-46b2-a5e2-88229988ff22",  
  "nombre": "Fernando Llancao",  
  "rol": "Administrador"  
},  
"expiraEnSegundos": 1800  
}
```

Respuesta de Error (401 Unauthorized):

```
{  
  "codigoError": "CREDENCIALES_INVALIDAS",  
  "mensaje": "El usuario o la contraseña ingresados son incorrectos.",  
  "timestamp": "2026-06-03T01:10:00Z"  
}
```

7.1.2. 2. Reserva Temporal de Stock (Inventory Service)

Endpoint: POST /api/v1/inventory/reserve

Descripción: Reserva temporalmente unidades de un producto en una ubicación específica para evitar la sobreventa desde el canal web.

Estructura de la Petición (Request):

```
{
  "ordenId": "d3b07384-d113-49c5-a5d6-d68aee460183",
  "items": [
    {
      "sku": "PES-CANA-CARBON-01",
      "cantidad": 2,
      "ubicacionId": "e2a22288-e22a-43c2-bf72-888e422201b1"
    }
  ]
}
```

Respuesta de Éxito (201 Created):

```
{
  "reservaId": "b1b88220-432a-4632-a5e2-88229988ff22",
  "estado": "Pendiente",
  "fechaCreacion": "2026-06-03T01:10:00Z",
  "fechaExpiracion": "2026-06-03T01:25:00Z"
}
```

Respuesta de Error - Stock Insuficiente (409 Conflict):

```
{
  "codigoError": "STOCK_INSUFICIENTE",
  "mensaje": "No hay suficiente stock para el producto indicado en esa bodega.",
  "timestamp": "2026-06-03T01:10:01Z",
  "detalles": {
    "sku": "PES-CANA-CARBON-01",
    "stockDisponible": 1,
    "cantidadSolicitada": 2
  }
}
```

7.1.3. 3. Confirmación de Reserva (Inventory Service)

Endpoint: POST /api/v1/inventory/commit

Descripción: Transforma una reserva temporal en una venta confirmada y descontada definitivamente.

Estructura de la Petición (Request):

```
{
  "reservaId": "b1b88220-432a-4632-a5e2-88229988ff22",
  "ordenId": "d3b07384-d113-49c5-a5d6-d68aee460183"
}
```

Respuesta de Éxito (200 OK):

```
{
  "status": "success",
  "mensaje": "Reserva confirmada e inventario rebajado definitivamente.",
  "transaccionId": "c8e228c2-482a-466d-88b1-2299aaffd883"
}
```

Respuesta de Error - Reserva Expirada o Inexistente (410 Gone):

```
{
  "codigoError": "RESERVA_NO_VALIDA",
  "mensaje": "La reserva no existe, ya fue consolidada o expiró tras el límite de 15 m",
  "timestamp": "2026-06-03T01:10:05Z",
  "detalles": {
    "reservaId": "b1b88220-432a-4632-a5e2-88229988ff22",
    "estadoActual": "Expirada"
  }
}
```

7.1.4. 4. Consulta de Stock en Tiempo Real (Inventory Service)

Endpoint: GET /api/v1/inventory/stock/{sku}

Descripción: Devuelve la disponibilidad detallada de un SKU en todas las ubicaciones físicas y bodegas registradas.

Respuesta de Éxito (200 OK):

```
{
  "sku": "PES-CANA-CARBON-01",
  "nombre": "Caña de Pescar Carbono Pro",
  "precio": 45000.00,
  "inventario": [
    {
      "ubicacionId": "e2a22288-e22a-43c2-bf72-888e422201b1",
      "ubicacionNombre": "Tienda Santiago POS",
      "stockDisponible": 15,
      "stockReservado": 2,
      "stockTransito": 0
    },
    {
      "ubicacionId": "a5d3f288-43ea-46b2-a5e2-88229988abcd",
      "ubicacionNombre": "Bodega Principal E-commerce",

```

```
    "stockDisponible": 50,  
    "stockReservado": 8,  
    "stockTransito": 10  
  }  
]  
}
```

8. Documentación de Servicios y Procesos Cliente

Para soportar las nuevas funcionalidades e integraciones de la 3ra Entrega, se ha extendido el contrato técnico del sistema Sync-Stock. Las peticiones externas son recibidas por el router de Express (actuando como un Protocol Bridge), el cual decodifica y valida las credenciales y el token JWT, evalúa el control de acceso por rol (RBAC) con el middleware `permitirRoles`, empaqueta el contenido y lo retransmite al Enterprise Service Bus (ESB) por sockets TCP en formato JSON.

A continuación se detallan las operaciones adicionales incorporadas al catálogo de servicios:

8.1. Registro de Nuevos Productos

Endpoint: POST `/api/v1/sales/products`

Rol Autorizado: Admin

Descripción: Registra un nuevo producto en el catálogo y de forma automática crea un registro de inventario con stock inicial en todas las ubicaciones y bodegas disponibles bajo una única transacción ACID.

Estructura de la Petición (Request):

```
{  
  "sku": "PES-SE-NZ-023",  
  "nombre": "Señuelo Rapala Magnum",  
  "precio": 12500,  
  "stockInicial": 20  
}
```

Respuesta de Éxito (201 Created):

```
{  
  "status": "success",  
  "mensaje": "Producto registrado y stock inicial sembrado.",  
  "producto": {  
    "id": "f5b88220-432a-4632-a5e2-88229988ffaa",  
    "sku": "PES-SE-NZ-023",  
    "nombre": "Señuelo Rapala Magnum",  
    "precio": 12500  
  }  
}
```


Respuesta de Error - SKU Duplicado (400 Bad Request):

```
{
  "codigoError": "SKU_DUPLICADO",
  "mensaje": "El SKU proporcionado ya está registrado en el sistema.",
  "timestamp": "2026-06-03T19:30:00Z"
}
```

8.2. Modificación de Producto

Endpoint: PUT /api/v1/sales/products/:sku

Rol Autorizado: Admin

Descripción: Actualiza los atributos básicos (nombre y precio) de un producto existente identificado por su SKU en el catálogo.

Estructura de la Petición (Request):

```
{
  "nombre": "Señuelo Rapala Magnum Pro",
  "precio": 13990
}
```

Respuesta de Éxito (200 OK):

```
{
  "status": "success",
  "mensaje": "Producto actualizado exitosamente.",
  "producto": {
    "sku": "PES-SE-NZ-023",
    "nombre": "Señuelo Rapala Magnum Pro",
    "precio": 13990
  }
}
```

8.3. Despacho de Órdenes Consolidadas

Endpoint: POST /api/v1/sales/orders/:id/dispatch

Rol Autorizado: Admin, ECommerce

Descripción: Marca una orden pagada y reservada temporalmente como despachada físicamente, actualizando su estado interno y liberando la reserva del flujo activo.

Respuesta de Éxito (200 OK):

```
{
  "status": "success",
  "mensaje": "Orden d3b07384 despachada exitosamente."
}
```

Respuesta de Error - Estado Inválido (400 Bad Request):

```
{
  "codigoError": "ESTADO_DESPACHO_INVALIDO",
  "mensaje": "Solo se pueden despachar órdenes que estén en estado Pagada.",
  "timestamp": "2026-06-03T19:35:00Z"
}
```

8.4. Redirección General a través del Bus de Servicios (GET)

Para dar cumplimiento estricto con el diseño arquitectónico SOA definido para PescaTotal, se refactorizaron las rutas del módulo de ventas en el router REST del backend. Peticiones de solo lectura como:

- GET /sales/products (Catálogo completo de productos)
- GET /sales/orders (Historial consolidado de órdenes)
- GET /sales/reservations (Listado de reservas web activas)
- GET /sales/history (Log de auditoría de movimientos)

Ya no realizan llamadas directas de lectura a Prisma en el servidor web Express. En su lugar, emiten una llamada TCP estructurada a través de `requestSOABus` al microservicio correspondiente. El bus de servicios ESB se encarga de resolver la consulta y retornar la respuesta, manteniendo el desacoplamiento físico de la capa de datos.

9. Documentación del Sistema y Evidencias de Funcionamiento

9.1. Explicación Técnica del Código e Implementación

El sistema Sync-Stock se implementó utilizando Node.js, TypeScript y Prisma, logrando los objetivos mediante los siguientes componentes core:

1. **Bus de Servicios TCP (ESB Customizado):** Implementado en `soabus.ts`, este módulo levanta un socket TCP en el puerto 5000. Los microservicios de inventario (`inven`), ventas (`sales`) y autenticación (`autho`) se registran abriendo sockets persistentes. La mensajería interna utiliza un protocolo de entramado customizado de longitud fija (Length-Prefixed Framing). Cada mensaje enviado posee un prefijo de 5 caracteres numéricos que especifican el tamaño exacto del contenido en bytes, seguido de 5 caracteres que identifican al servicio de destino. Esto previene problemas clásicos de fragmentación de buffer en streams TCP socket concurrentes.
2. **Control de Concurrencia y Bloqueos Pesimistas:** Para garantizar que no ocurran sobreventas en checkout simultáneos de POS físico y reservas de E-commerce, el servicio de inventario (`inventory.ts`) utiliza transacciones pesimistas (`prisma.$transaction`). Dentro de la transacción, se ejecuta una consulta raw SQL utilizando `SELECT * FROM 'Inventario'... FOR UPDATE`. Esto bloquea temporalmente las filas de stock específicas de los productos requeridos, forzando a que cualquier transacción concurrente deba esperar a que la transacción actual confirme (`commit`) o aborte (`rollback`).
3. **Prevención de Deadlocks:** Para evitar el problema de bloqueo mutuo (*deadlocks*) donde dos transacciones concurrentes retienen recursos cruzados, se incorporó un algoritmo de ordenamiento determinista en la función de checkout. Antes de adquirir los bloqueos en base de datos, el backend ordena el array de productos por su UUID de forma ascendente. Al bloquear los recursos siempre en el mismo orden físico en el disco, se imposibilita la formación de ciclos de espera.
4. **Daemon de Expiración (TTL) en Background:** En `backend/src/index.ts` se configuró un proceso daemon en segundo plano que corre cada 10 segundos. Este daemon escanea la base de datos de manera atómica buscando registros en la tabla `Reserva` que estén en estado `Pendiente` y cuya `fechaExpiracion` sea menor al tiempo actual. Al detectar expiraciones, abre una transacción que revierte las unidades reservadas devolviéndolas al stock vendible disponible y actualiza el estado a `Expirada`.

9.2. Evidencias de Funcionamiento y Logs

A continuación se adjuntan las trazas de ejecución en consola obtenidas directamente del servidor de backend de Sync-Stock, demostrando el ciclo de vida de los servicios, el control de concurrencia y las tareas en segundo plano.

Logs de Inicialización del ESB y Conexión de Servicios:

```
[ESB-TCP] Server listening on port 5000...
[ESB-TCP] New client connection established.
[ESB-TCP] Registered service: 'autho' on local socket.
[ESB-TCP] Registered service: 'inven' on local socket.
[ESB-TCP] Registered service: 'sales' on local socket.
[ESB-Mediator] ESB Mediator registering services via TCP sockets...
[ESB-Mediator] Services successfully mapped and listening for requests.
[Express-Gateway] Gateway API listening on port 3001.
```

Logs de Transacción de POS y Bloqueo Pesimista (Checkout Concurrente):

```
[Express-Gateway] POST /api/v1/sales/checkout received from POS
[ESB-TCP] Routing message to service 'sales' (action: 'checkout')
[Sales-Service] Processing checkout. Total items: 2.
[Inventory-Service] Initiating stock check and lock transaction.
[Inventory-Service] Sorted product list to prevent deadlocks:
  - SKU: PES-CANA-CARBON-01 (UUID: a1b2c3d4-...)
  - SKU: PES-SE-NZ-023 (UUID: f5b88220-...)
[Prisma-SQL] SELECT * FROM "Inventario" WHERE ... FOR UPDATE
[Prisma-SQL] Row locks acquired successfully. No contention.
[Inventory-Service] Stock verified. Deducting 2 units from available stock.
[Prisma-SQL] UPDATE "Inventario" SET "stockDisponible" = 13 WHERE ...
[Sales-Service] Order d3b07384-d113... recorded successfully.
[Inventory-Service] Transaction committed. Locks released.
[ESB-TCP] Transaction checkout response returned to Protocol Bridge.
[Express-Gateway] 201 Created returned to POS vendor.
```

Logs del Daemon de Expiración Temporal de Reservas Web (TTL):

```
[Daemon-TTL] Scanning database for expired web reservations...
[Daemon-TTL] Found 1 expired pending reservation (ID: b1b88220-432a...).
[Inventory-Service] Initiating reservation rollback transaction.
[Prisma-SQL] SELECT * FROM "Inventario" WHERE ... FOR UPDATE
[Inventory-Service] Reverting 2 units of SKU PES-CANA-CARBON-01.
[Prisma-SQL] UPDATE "Inventario" SET
  "stockDisponible" = "stockDisponible" + 2,
  "stockReservado" = "stockReservado" - 2 WHERE ...
[Prisma-SQL] UPDATE "Reserva" SET "estado" = 'Expirada' WHERE ...
[Inventory-Service] Rollback transaction committed successfully.
[Daemon-TTL] Expired reservation b1b88220 reclaimed. Stock restored.
```

10. Análisis Crítico

10.1. Latencia y Concurrencia de los Bloqueos Pesimistas

El uso de bloqueos pesimistas (`SELECT ... FOR UPDATE`) garantiza una consistencia absoluta (consistencia estricta en el contexto del teorema de CAP), lo cual es crítico para erradicar las sobreventas en PescaTotal. Sin embargo, esta decisión de diseño conlleva un costo en latencia bajo escenarios de alta concurrencia. Cuando se bloquean filas de inventario para un producto altamente demandado (por ejemplo, ofertas flash en el canal web), las peticiones de checkout concurrentes se encolan secuencialmente en el motor PostgreSQL, aumentando la latencia general del sistema de < 50 ms hasta picos de > 1000 ms. Para mitigar esto en un entorno productivo de mayor escala, se debería evaluar una estrategia híbrida utilizando optimismo de stock mediante caches en memoria (ej. Redis) o reservas distribuidas desacopladas del motor de base de datos relacional.

10.2. Desafíos del Desarrollo de un ESB Personalizado TCP

La decisión de implementar un bus de servicios personalizado sobre sockets TCP crudos en Node.js permitió lograr un sistema ultraligero y sin dependencias de infraestructura externas. No obstante, en un entorno de desarrollo real esto introduce complejidades de bajo nivel que los brokers industriales (como RabbitMQ, Apache Kafka o gRPC) resuelven de forma nativa:

- **Entramado de Datos y Fragmentación:** Al usar streams TCP crudos, los mensajes pueden llegar fragmentados o concatenados si se transmiten de forma muy veloz. Se tuvo que programar un algoritmo de parseo manual con prefijo de longitud para solucionar esto de forma segura.
- **Gestión de Colas y Reintentos:** A diferencia de RabbitMQ, nuestro bus customizado no almacena físicamente los mensajes en disco (encolamiento persistente) ni cuenta con mecanismos de Dead Letter Queues (DLQ) para reenvío de mensajes fallidos de forma automática.

10.3. Seguridad y RBAC en Entornos Distribuidos

El diseño e implementación de un middleware de control de acceso por roles (RBAC) centralizado en el Gateway REST protege con éxito la capa lógica del ESB de peticiones externas no autorizadas. Los usuarios del sistema solo interactúan con pantallas permitidas (por ejemplo, restringiendo la pestaña de Catálogo CRUD y la Auditoría de mensajería del ESB al administrador). Sin embargo, a nivel de red interna, la comunicación TCP entre el Protocol Bridge (Express) y el bus de servicios viaja actualmente en texto plano. En un entorno de producción distribuido real en la nube, es imperativo implementar encriptación de extremo a extremo mediante el módulo `tls` de Node.js, configurando certificados SSL/TLS autofirmados para mitigar ataques de escucha de red interna (Man-in-the-Middle).

10.4. Control de Inactividad y Seguridad Física en Tienda

La incorporación de la autodetección de inactividad de 30 minutos es un paso crítico para proteger la seguridad de la información física en la tienda PescaTotal. Dado que las

terminales del Punto de Venta (POS) suelen compartirse entre diferentes vendedores en el mesón de atención física, un olvido de cierre de sesión podría permitir a un vendedor registrar ventas a nombre de otro o a un cliente acceder a configuraciones de stock del panel administrador. La simulación de inactividad implementada proporciona un mecanismo de auditoría rápido que valida que el estado reactivo del frontend se limpie correctamente al expirar la sesión.

11. Conclusiones

Con la definición del modelo de datos relacional y la estructuración en base a servicios (SOA), el sistema **Sync-Stock** adquiere la solidez técnica necesaria para erradicar la sobreventa en PescaTotal. La separación entre componentes cliente y servicio asegura que la plataforma sea escalable, permitiendo a futuro integrar nuevos canales de venta sin alterar el núcleo del control de inventario. La estricta vinculación de cada interfaz con los requerimientos de seguridad, consistencia y baja latencia asegura el cumplimiento tanto de los objetivos comerciales como de los estándares de ingeniería requeridos.